

Code Meeting: Einstein Toolkit

Alex Vañó Viñuales

CENTRA, Instituto Superior Técnico, Universidade de Lisboa

GRIT Code Meeting

25 February 2025

Einstein Toolkit

What is it? Collection of thorns based on **Cactus** code for NR.

Einstein Toolkit

What is it? Collection of thorns based on **Cactus** code for NR.

The “flesh” are the necessary components of the code that allow to use the “thorns” – the specific modules that do something in particular.

Einstein Toolkit

What is it? Collection of thorns based on **Cactus** code for NR.

The “**flesh**” are the necessary components of the code that allow to use the “**thorns**” – the specific modules that do something in particular.

Crash course on the Einstein Toolkit by Gabriele Bozzola 2 years ago:
<https://youtu.be/1tAvpWyanX8>

List with further tutorials: <https://www.youtube.com/playlist?list=PLIVVtc6RlFxrKU800u-bpmaGwDk09Xqpy>

Compile code to later run multipatch example

How long did it take you to compile the Einstein Toolkit?

Compile code to later run multipatch example

How long did it take you to compile the Einstein Toolkit?

Compile only with the needed thorns (faster):

Go to Cactus folder.

Create thornlist with only the thorns in the parfile (Kerr-Schild_Multipole.par from <https://einsteintoolkit.org/gallery/multipatch/index.html>):

```
utils/Scripts/MakeThornList -m thornlists/einsteintoolkit.th  
par/Kerr-Schild_Multipole.par > thornlists/kerrschildmultipole.th
```

Compile code to later run multipatch example

How long did it take you to compile the Einstein Toolkit?

Compile only with the needed thorns (faster):

Go to Cactus folder.

Create thornlist with only the thorns in the parfile (Kerr-Schild_Multipole.par from <https://einsteintoolkit.org/gallery/multipatch/index.html>):

```
utils/Scripts/MakeThornList -m thornlists/einsteintoolkit.th  
par/Kerr-Schild_Multipole.par > thornlists/kerrschildmultipole.th
```

Build/compile “ksmultipole”:

```
./simfactory/bin/sim build ksmultipole -j2 --thornlist  
thornlists/kerrschildmultipole.th
```

Compile code to later run multipatch example

How long did it take you to compile the Einstein Toolkit?

Compile only with the needed thorns (faster):

Go to Cactus folder.

Create thornlist with only the thorns in the parfile (Kerr-Schild_Multipole.par from <https://einstein toolkit.org/gallery/multipatch/index.html>):

```
utils/Scripts/MakeThornList -m thornlists/einsteintoolkit.th  
par/Kerr-Schild_Multipole.par > thornlists/kerrschildmultipole.th
```

Build/compile “ksmultipole”:

```
./simfactory/bin/sim build ksmultipole -j2 --thornlist  
thornlists/kerrschildmultipole.th
```

And while we wait ...

General code structure and philosophy

Thorns written in different programming languages.

Check list of thorns in

<https://einsteintoolkit.org/documentation/ThornGuide.php>

Thorn structure

Parameter file

simfactory: create, run, list, stop, purge runs ...

Overview of the code's capabilities

Initial data thorns

Evolution thorns

MHD thorns

Extra field thorns

Analysis thorns (apparent horizon finder)

GW extraction thorns

Comparison with other codes

- SpEC / SpECTRE
- bamps
- GRChombo $\rightarrow$ GRTeclyn
- MHDuet
-

Run multipatch example

Create simulation:

```
./simfactory/bin/sim create Kerr-Schild_Multipole --configuration  
ksmultipole --parfile par/Kerr-Schild_Multipole.par
```

Run multipatch example

Create simulation:

```
./simfactory/bin/sim create Kerr-Schild_Multipole --configuration  
ksmultipole --parfile par/Kerr-Schild_Multipole.par
```

Check existing simulations:

Run multipatch example

Create simulation:

```
./simfactory/bin/sim create Kerr-Schild_Multipole --configuration  
ksmultipole --parfile par/Kerr-Schild_Multipole.par
```

Check existing simulations:

```
./simfactory/bin/sim list-simulations
```

Run multipatch example

Create simulation:

```
./simfactory/bin/sim create Kerr-Schild_Multipole --configuration  
ksmultipole --parfile par/Kerr-Schild_Multipole.par
```

Check existing simulations:

```
./simfactory/bin/sim list-simulations
```

Submit simulation:

```
./simfactory/bin/sim submit Kerr-Schild_Multipole --cores=2  
--num-threads=1 --walltime=0:20:00
```

Run multipatch example

Create simulation:

```
./simfactory/bin/sim create Kerr-Schild_Multipole --configuration  
ksmultipole --parfile par/Kerr-Schild_Multipole.par
```

Check existing simulations:

```
./simfactory/bin/sim list-simulations
```

Submit simulation:

```
./simfactory/bin/sim submit Kerr-Schild_Multipole --cores=2  
--num-threads=1 --walltime=0:20:00
```

Follow code's output as it's being written:

```
./simfactory/bin/sim show-output --follow Kerr-Schild_Multipole
```

Run multipatch example

Create simulation:

```
./simfactory/bin/sim create Kerr-Schild_Multipole --configuration  
ksmultipole --parfile par/Kerr-Schild_Multipole.par
```

Check existing simulations:

```
./simfactory/bin/sim list-simulations
```

Submit simulation:

```
./simfactory/bin/sim submit Kerr-Schild_Multipole --cores=2  
--num-threads=1 --walltime=0:20:00
```

Follow code's output as it's being written:

```
./simfactory/bin/sim show-output --follow Kerr-Schild_Multipole
```

Basic commands in <https://github.com/einsteintoolkit/jupyter-et/blob/master/tutorial-server/notebooks/CactusTutorial.ipynb>

Visualization of output

Use fast tools for checking runs (kuibit, ...).

Use Paraview or VisIt for cool plots.

Let's create a thorn with Kranc and compile!

```
cd Kranc/Examples
```

```
../../Kranc/Bin/kranc SimpleWave.m
```

Let's create a thorn with Kranc and compile!

```
cd Kranc/Examples  
../../Kranc/Bin/kranc SimpleWave.m
```

Let's add the Krank thorns to the thornlist:

```
cp einsteintoolkit.th kranc.th  
echo 'KrancNumericalTools/GenericFD KrancExamples/SimpleWave' >>  
kranc.th
```

Check available code configurations (basically, compiled executables):

```
./simfactory/bin/sim list-configurations
```

Let's create a thorn with Kranc and compile!

```
cd Kranc/Examples  
../../Kranc/Bin/kranc SimpleWave.m
```

Let's add the Krank thorns to the thornlist:

```
cp einsteintoolkit.th kranc.th  
echo 'KrancNumericalTools/GenericFD KrancExamples/SimpleWave' >>  
kranc.th
```

Check available code configurations (basically, compiled executables):

```
./simfactory/bin/sim list-configurations
```

Let's compile Cactus only with the thorns used in the relevant parameter file:

```
utils/Scripts/MakeThornList -m thornlists/kranc.th  
../../Kranc/Examples/simplewave_sine.par >  
thornlists/kranc_simplewave_sine.th
```

Let's create a thorn with Kranc and compile!

```
cd Kranc/Examples  
../../Kranc/Bin/kranc SimpleWave.m
```

Let's add the Krank thorns to the thornlist:

```
cp einsteintoolkit.th kranc.th  
echo 'KrancNumericalTools/GenericFD KrancExamples/SimpleWave' >>  
kranc.th
```

Check available code configurations (basically, compiled executables):

```
./simfactory/bin/sim list-configurations
```

Let's compile Cactus only with the thorns used in the relevant parameter file:

```
utils/Scripts/MakeThornList -m thornlists/kranc.th  
../../Kranc/Examples/simplewave_sine.par >  
thornlists/kranc_simplewave_sine.th
```

Build/compile “kranc_test”:

```
./simfactory/bin/sim build kranc_test -j2 --thornlist  
thornlists/kranc_simplewave_sine.th
```

Run Kranc generated thorn

Create simulation setup:

```
./simfactory/bin/sim create kranc_sine --configuration kranc_test  
--parfile ../Kranc/Examples/simplewave_sine.par
```

Run Kranc generated thorn

Create simulation setup:

```
./simfactory/bin/sim create kranc_sine --configuration kranc_test  
--parfile ../Kranc/Examples/simplewave_sine.par
```

Submit run:

```
./simfactory/bin/sim submit kranc_sine --cores=2 --num-threads=1  
--walltime=0:20:00
```

Check status of run:

```
./simfactory/bin/sim list-simulations kranc_sine
```

Run Kranc generated thorn

Create simulation setup:

```
./simfactory/bin/sim create kranc_sine --configuration kranc_test  
--parfile ../Kranc/Examples/simplewave_sine.par
```

Submit run:

```
./simfactory/bin/sim submit kranc_sine --cores=2 --num-threads=1  
--walltime=0:20:00
```

Check status of run:

```
./simfactory/bin/sim list-simulations kranc_sine
```

Follow code's output as it's being written:

```
./simfactory/bin/sim show-output --follow kranc_sine
```

Extending the code

You can create your own thorns or modify existing ones.

Code comparison

	EINSTEIN TOOLKIT	SPECTRE
Gravity discretization	FD	PSC
Formulations	BSSN / Z4c?	GHG
Singularities	Moving puncture	Excision
Moving black holes	✓	✓
AMR	✓	✓
Grid structure	Cube or Multipatch*	Multipatch
Parallelization	MPI/GPU	Charm++
AH finder	✓	✓
Elliptic solver formulation	Two Punctures**	XCTS
Elliptic solver method	Single domain PSC	Multigrid
Gravitational wave extraction	Extraction at finite radius	CCE

*Not available for the most recent driver, CarpetX.

**CTT-like method with conformally flat, maximally sliced vacuum initial data, and a given puncture analytical solution for the momentum constraint.